

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Проектирование защищённых интеллектуальных
информационных систем»
на тему:

Управление доступом с помощью списков контроля доступа.

**Часть 1. Управление доступом к объектам операционных
систем.**

**Часть 2. Управление доступом к приложениям и системам
базам данных**

Студенты гр. 321701

Руководитель

И. А. Политыко
В. А. Лукашов
Д. А. Сальников

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	3
1 Постановка задачи	4
2 Часть 1. Управление доступом к объектам операционных систем	5
3 Часть 2. Управление доступом к приложениям и системам ба- зам данных	8
Заключение	10
Список использованных источников	11

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ACL	— Access Control List, Список контроля доступа;
ACE	— Access Control Entry, Запись списка контроля доступа;
DAC	— Discretionary Access Control, Избирательное управление доступом;
ОС	— Операционная система;
СУБД	— Система управления базами данных;
БД	— База данных;
SQL	— Structured Query Language, Язык структурированных запросов;
DDL	— Data Definition Language, Язык определения данных;
DML	— Data Manipulation Language, Язык манипулирования данными;
CRUD	— Create, Read, Update, Delete, Базовые операции над данными;
GRANT / REVOKE	— Команды назначения и отзыва привилегий в СУБД;
rwX	— Права чтения (read), записи (write) и исполнения (execute);
UID / GID	— Идентификаторы пользователя и группы в UNIX-подобных ОС.

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение механизмов избирательного управления доступом: на уровне объектов файловой системы операционной системы Linux и на уровне ролей и привилегий в системе управления базами данных PostgreSQL.

В части 1 требуется разработать программу (набор сценариев командной оболочки), которая создаёт группы пользователей `group_iit1`, `group_iit2` и пользователей `iit11`, `iit12`, `iit21`, `iit22`, `iit3` с указанным членством в группах, а также предоставляет пользователю `iit21` административные привилегии; создаёт каталог `/home/pzs` и подкаталоги `pzs11–pzs15` с правами доступа только для владельца, только для группы, только для остальных, для всех пользователей и только для администратора (`root`); от имени пользователя `iit11` создаёт в доступных каталогах набор файлов `file11–file55` с комбинациями прав чтения, записи и исполнения; проверяет для указанных пользователей и суперпользователя возможность чтения, изменения и исполнения файлов, остановки процессов, запущенных по шаблону `filex5`, а также чтения содержимого каталогов, создания и удаления файлов в них; удаляет созданные файлы, каталоги, пользователей и группы.

В части 2 (вариант — PostgreSQL) требуется установить СУБД, настроить механизм контроля доступа с ролями администратора (полный доступ), пользователя (CRUD) и гостя (только чтение), проверить корректность политики безопасности и удалить созданные объекты СУБД.

2 ЧАСТЬ 1. УПРАВЛЕНИЕ ДОСТУПОМ К ОБЪЕКТАМ ОПЕРАЦИОННЫХ СИСТЕМ

Решение реализовано набором сценариев `bash` в каталоге `lab1/p1`. Назначение скриптов приведено в таблице 2.1.

Скрипт	Назначение
<code>setup.sh</code>	Создание групп, пользователей, каталогов и файлов (п. 1–13)
<code>verify_files.sh</code>	Проверка чтения, записи и исполнения файлов (п. 14)
<code>verify_procs.sh</code>	Запуск файлов шаблона <code>filex5</code> и проверка <code>kill</code> (п. 15)
<code>verify_dirs.sh</code>	Проверка чтения каталогов, создания и удаления файлов (п. 16)
<code>cleanup.sh</code>	Удаление стенда (п. 17)

Таблица 2.1 — Скрипты части 1

В модели DAC UNIX-подобных систем права владельца проверяются раньше прав группы. Поэтому каталог с режимом «только для группы» (070) нельзя отдавать пользователю `iit11` как владельцу: иначе у него окажутся нулевые биты владельца, и доступ по группе не сработает. Аналогично для режима «только остальные» (007). Принятая схема владельцев каталогов приведена в таблице 2.2.

Каталог	Режим	Владелец:группа	Смысл
<code>pzs11</code>	700	<code>iit11:group_iit1</code>	только владелец
<code>pzs12</code>	070	<code>root:group_iit1</code>	только группа
<code>pzs13</code>	007	<code>root:root</code>	только остальные
<code>pzs14</code>	777	<code>iit11:group_iit1</code>	все пользователи
<code>pzs15</code>	700	<code>root:root</code>	только администратор

Таблица 2.2 — Каталоги `/home/pzs` и права доступа

Содержимое файлов соответствует методичке: для шаблона `filex5` (`file15`, `file25`, ..., `file55`) — строка `read testVariable`; для остальных — `echo "Hello World"`. Файлы `file51`–`file55` передаются администратору (`chown root:root`). Проверки доступа неразрушающие: запись в файл выполняется с откатом содержимого; в каталогах создаётся и удаляется только зонд `.__probe_*`.

Порядок запуска:

```
1 cd lab1/p1
2 chmod +x *.sh
```

```

3 sudo ./setup.sh
4 sudo ./verify_files.sh
5 sudo ./verify_procs.sh
6 sudo ./verify_dirs.sh
7 # sudo ./cleanup.sh

```

Листинг 2.1— Запуск сценариев части 1

Результаты сохраняются в каталоге `lab1/p1/results/` (*.txt, *.tsv). Экспериментальный прогон выполнен 10.09.2026 на хосте с ОС Linux.

Фрагмент проверки каталогов (п. 16) приведён в таблице 2.3 (значение «ДА» означает успешные list, create и delete зонда).

Пользователь	pzs11	pzs12	pzs13	pzs14	pzs15	
iit11	ДА	ДА	ДА	ДА	НЕТ	
iit12	НЕТ	ДА	ДА	ДА	НЕТ	
iit21	НЕТ	НЕТ	ДА	ДА	НЕТ	
iit22	НЕТ	НЕТ	ДА	ДА	НЕТ	
iit3	НЕТ	НЕТ	ДА	ДА	НЕТ	
root	ДА	ДА	ДА	ДА	ДА	

Таблица 2.3— Доступ к каталогам (list / create / delete), 10.09.2026

Результаты п. 15 (остановка процессов `filex5`, запущенных от `iit11`) приведены в таблице 2.4.

Файл	iit11	iit12	iit21	iit22	iit3	root	
file15	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file25	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file35	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file45	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	
file55	ДА	НЕТ	НЕТ	НЕТ	НЕТ	ДА	

Таблица 2.4— Возможность *kill* процесса `filex5` (п. 15)

Фрагмент проверки файлов (п. 14) для пользователя `iit11` в каталоге `pzs11` — таблица 2.5.

Ключевые фрагменты сценария настройки приведены в листингах 2.2 и 2.3.

```

1 chown iit11:group_iit1 "${BASE}/pzs11"; chmod 700 "${BASE}/pzs11"
2 chown root:group_iit1 "${BASE}/pzs12"; chmod 070 "${BASE}/pzs12"
3 chown root:root "${BASE}/pzs13"; chmod 007 "${BASE}/pzs13"
4 chown iit11:group_iit1 "${BASE}/pzs14"; chmod 777 "${BASE}/pzs14"
5 chown root:root "${BASE}/pzs15"; chmod 700 "${BASE}/pzs15"

```

Листинг 2.2— Владельцы и режимы каталогов

Файл	R	W	X	Комментарий
file11	ДА	НЕТ	НЕТ	только чтение владельца
file12	ДА	ДА	НЕТ	чтение и запись владельца
file13	НЕТ	ДА	НЕТ	только запись владельца
file14	ДА	ДА	ДА	rwX владельца
file15	НЕТ	НЕТ	ДА	только исполнение владельца
file21–file25	НЕТ	НЕТ	НЕТ	владелец; биты owner=0
file41	ДА	НЕТ	НЕТ	права «для всех»
file44	ДА	ДА	ДА	права «для всех»
file51–file55	НЕТ	НЕТ	НЕТ	владелец root

Таблица 2.5— Доступ *iit11* к файлам в *pzs11* (READ / WRITE / EXEC)

```

1 for d in pzs11 pzs12 pzs13 pzs14; do
2     runuser -u iit11 -- "${CREATE_HELPER}" "${BASE}/${d}"
3 done
4 # file51..file55 -> root
5 chown root:root "${BASE}/${d}/file5"{1,2,3,4,5}

```

Листинг 2.3— Создание файлов от имени *iit11*

3 ЧАСТЬ 2. УПРАВЛЕНИЕ ДОСТУПОМ К ПРИЛОЖЕНИЯМ И СИСТЕМАМ БАЗАМ ДАННЫХ

PostgreSQL — объектно-реляционная система управления базами данных с развитой моделью ролей и привилегий (**GRANT/REVOKE**) на уровне баз данных, схем, таблиц, последовательностей и функций. Роли могут наследовать права других ролей, что позволяет отделить политику доступа (***_role**) от учётных записей входа (***_user**).

Скрипты части 2 расположены в каталоге `lab1/p2` (таблица 3.1).

Скрипт	Назначение
<code>install.sh</code>	Установка пакетов PostgreSQL и запуск службы
<code>setup_acl.sh</code>	Создание БД <code>lab_db</code> , схемы, ролей, таблицы и настройка <code>pg_hba.conf</code>
<code>verify.sh</code>	Матрица операций SELECT / INSERT / UPDATE / DELETE / CREATE / DROP
<code>cleanup.sh</code>	Удаление объектов лабы; ключ <code>--purge</code> удаляет пакеты СУБД

Таблица 3.1 — Скрипты части 2

В системе присутствуют три роли в соответствии с методичкой (таблица 3.2). Объекты стенда: база данных `lab_db`, схема `lab_schema`, таблица `lab_schema.test_table`.

Учётка	Пароль	Роль	Права
<code>admin_user</code>	<code>admin_pass</code>	<code>admin_role</code>	полный доступ к схеме (DML и DDL)
<code>app_user</code>	<code>user_pass</code>	<code>user_role</code>	SELECT , INSERT , UPDATE , DELETE
<code>guest_user</code>	<code>guest_pass</code>	<code>guest_role</code>	только SELECT

Таблица 3.2 — Роли и учётные записи PostgreSQL

Порядок запуска:

```
1 cd lab1/p2
2 chmod +x *.sh
3 sudo ./install.sh
4 sudo ./setup_acl.sh
5 ./verify.sh
6 # sudo ./cleanup.sh
```

Листинг 3.1 — Запуск сценариев части 2

Прогон выполнен 10.09.2026. Проверка политики безопасности завершилась без расхождений с ожиданиями (FAILS=0). Сводка — таблица 3.3; полный лог — lab1/p2/results/verify_acl.txt.

Учётка	SELECT	INSERT	UPDATE	DELETE	CREATE	DROP
admin_user	ДА	ДА	ДА	ДА	ДА	ДА
app_user	ДА	ДА	ДА	ДА	НЕТ	НЕТ
guest_user	ДА	НЕТ	НЕТ	НЕТ	НЕТ	—

Таблица 3.3— Результаты проверки ACL PostgreSQL, 10.09.2026

Таким образом, администратор имеет полный доступ, пользователь приложения — только DML (CRUD), гость — только чтение, что соответствует требованиям методички. Фрагмент назначения привилегий приведён в листинге 3.2.

```

1 GRANT ALL PRIVILEGES ON SCHEMA lab_schema TO admin_role;
2 GRANT CREATE ON SCHEMA lab_schema TO admin_role;
3 GRANT SELECT, INSERT, UPDATE, DELETE
4   ON ALL TABLES IN SCHEMA lab_schema TO user_role;
5 GRANT SELECT ON ALL TABLES IN SCHEMA lab_schema TO guest_role;
```

Листинг 3.2— Назначение привилегий ролям

ЗАКЛЮЧЕНИЕ

В рамках лабораторной работы №1 исследованы и программно реализованы механизмы избирательного управления доступом на двух уровнях: объектов файловой системы Linux и ролевой модели PostgreSQL. Экспериментальный прогон выполнен 10.09.2026.

Семантическая составляющая. Работа демонстрирует принципы ACL/DAC: в операционной системе — через классы «владелец / группа / остальные» и биты `rwX`, в СУБД — через роли и привилегии SQL. Показано, что эффективные права зависят не только от заданного набора битов или `GRANT`, но и от правил проверки доступа (в UNIX владелец не «проваливается» в групповые права) и от наследования ролей PostgreSQL.

Прагматическая составляющая. Подготовлен воспроизводимый стенд: автоматическое развёртывание, неразрушающая проверка политики и контролируемая очистка. Такой подход снижает риск ошибок ручной настройки и позволяет многократно демонстрировать и проверять ACL перед сдачей лабораторной.

Поставленные цели достигнуты: программы настройки и проверки доступа работоспособны; для PostgreSQL матрица операций дала `FAILS=0` (администратор — полный доступ, пользователь — CRUD без DDL, гость — только `SELECT`).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Олифер В. Г., Олифер Н. А. Сетевые операционные системы: учебник для вузов. — 2-е изд. — СПб. : Питер, 2009. — 669 с.
- [2] Таненбаум Э. С. Современные операционные системы. — 3-е изд. — СПб. : Питер, 2011. — 1120 с.
- [3] Ховард М., Лебланк Д. Защищённый код. — М. : Русская редакция, 2004. — 704 с.
- [4] CWE: Common Weakness Enumeration [Электронный ресурс]. — Режим доступа: <http://cwe.mitre.org/>. — Дата доступа: 10.09.2026.
- [5] PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>. — Дата доступа: 10.09.2026.
- [6] chmod(1) — Linux manual page [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man1/chmod.1.html>. — Дата доступа: 10.09.2026.